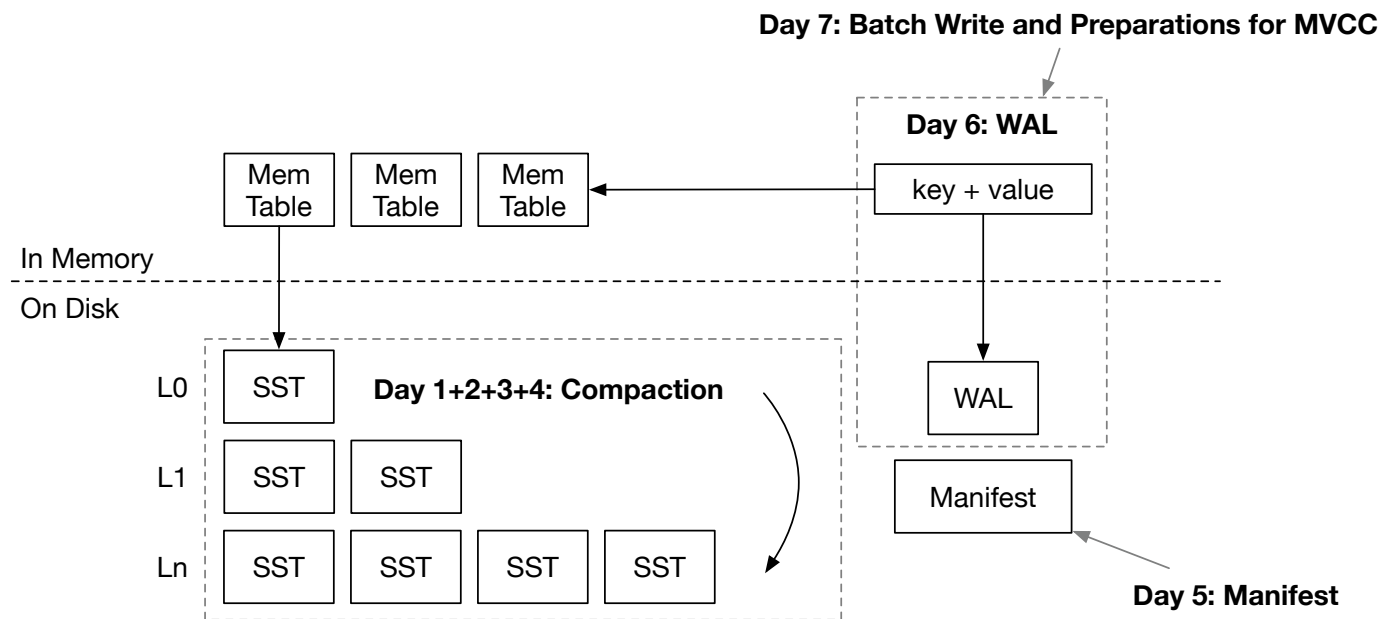


Week 2 Overview: Compaction and Persistence



In the last week, you have implemented all necessary structures for an LSM storage engine, and your storage engine already supports read and write interfaces. In this week, we will deep dive into the disk organization of the SST files and investigate an optimal way to achieve both performance and cost efficiency in the system. We will spend 4 days learning different compaction strategies, from the easiest to the most complex ones, and then implement the remaining parts for the storage engine persistence. At the end of this week, you will have a fully functional and efficient LSM storage engine.

We have 7 chapters (days) in this part:

- **Day 1: Compaction Implementation.** You will merge all L0 SSTs into a sorted run.
- **Day 2: Simple Leveled Compaction.** You will implement a classic leveled compaction algorithm and use compaction simulator to see how well it works.
- **Day 3: Tiered/Universal Compaction.** You will implement the RocksDB universal compaction algorithm and understand the pros/cons.
- **Day 4: Leveled Compaction.** You will implement the RocksDB leveled compaction algorithm. This compaction algorithm also supports partial compaction, so as to reduce peak space usage.
- **Day 5: Manifest.** You will store the LSM state on the disk and recover from the state.
- **Day 6: Write-Ahead Log (WAL).** User requests will be routed to both memtable and WAL so that all operations will be persisted.
- **Day 7: Write Batch and Checksums.** You will implement write batch API (for preparation for week 3 MVCC) and checksums for all of your storage formats.

Compaction and Read Amplification

Let us talk about compaction first. In the previous part, you simply flush the memtable to an L0 SST. Imagine that you have written gigabytes of data and now you have 100 SSTs. Every read request (without filtering) will need to read 100 blocks from these SSTs. This amplification is read amplification – the number of I/O requests you will need to send to the disk for one get operation.

To reduce read amplification, we can merge all the L0 SSTs into a larger structure, so that it would be possible to only read one SST and one block to retrieve the requested data. Say that we still have these 100 SSTs, and now, we do a merge sort of these 100 SSTs to produce another 100 SSTs, each of them contains non-overlapping key ranges. This process is **compaction**, and these 100 non-overlapping SSTs is a **sorted run**.

To make this process clearer, let us take a look at this concrete example:

```
SST 1: key range 00000 - key 10000, 1000 keys
SST 2: key range 00005 - key 10005, 1000 keys
SST 3: key range 00010 - key 10010, 1000 keys
```

We have 3 SSTs in the LSM structure. If we need to access key 02333, we will need to probe all of these 3 SSTs. If we can do a compaction, we might get the following 3 new SSTs:

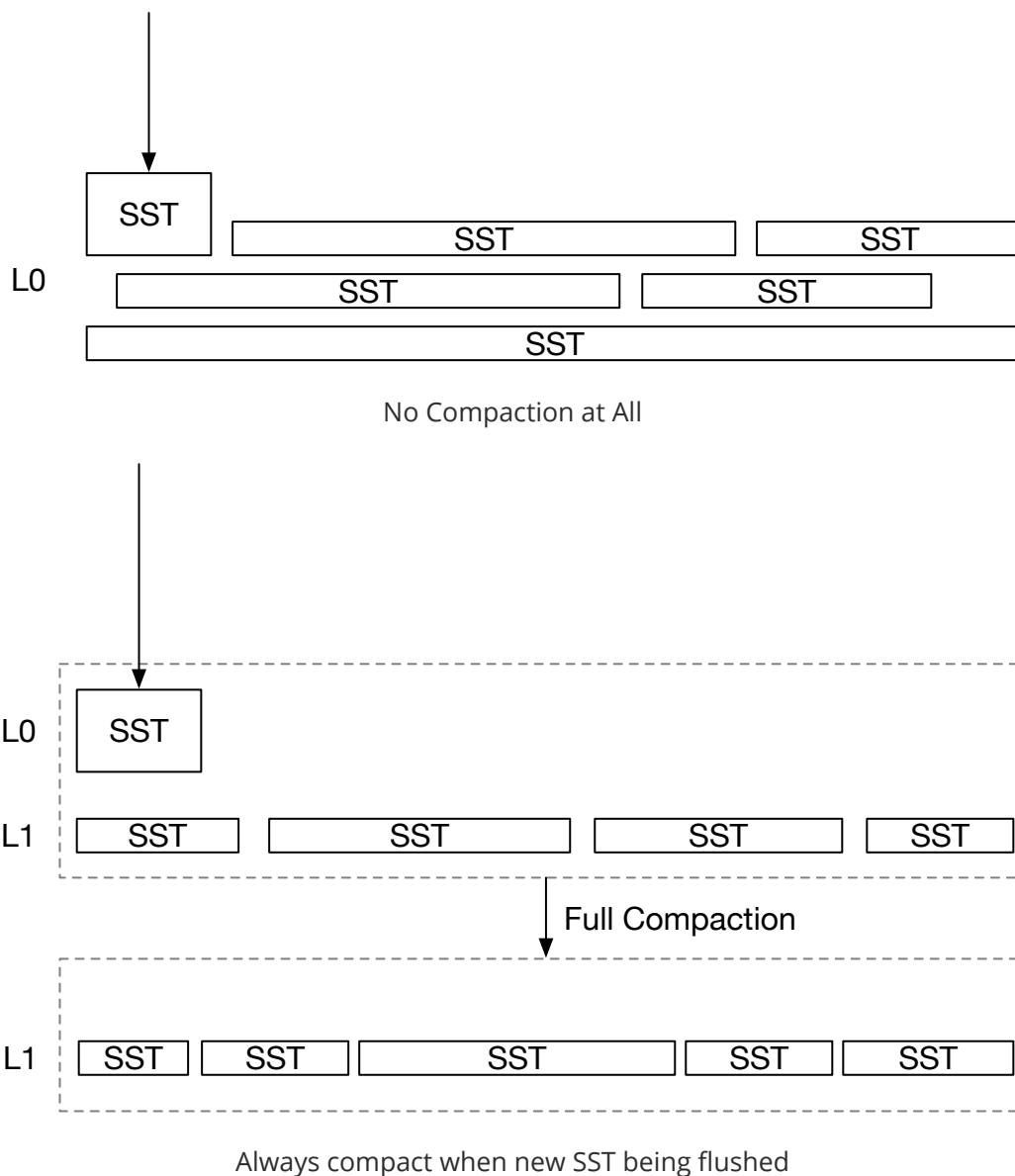
```
SST 4: key range 00000 - key 03000, 1000 keys
SST 5: key range 03001 - key 06000, 1000 keys
SST 6: key range 06000 - key 10010, 1000 keys
```

The 3 new SSTs are created by merging SST 1, 2, and 3. We can get a sorted 3000 keys and then split them into 3 files, so as to avoid having a super large SST file. Now our LSM state has 3 non-overlapping SSTs, and we only need to access SST 4 to find key 02333.

Two Extremes of Compaction and Write Amplification

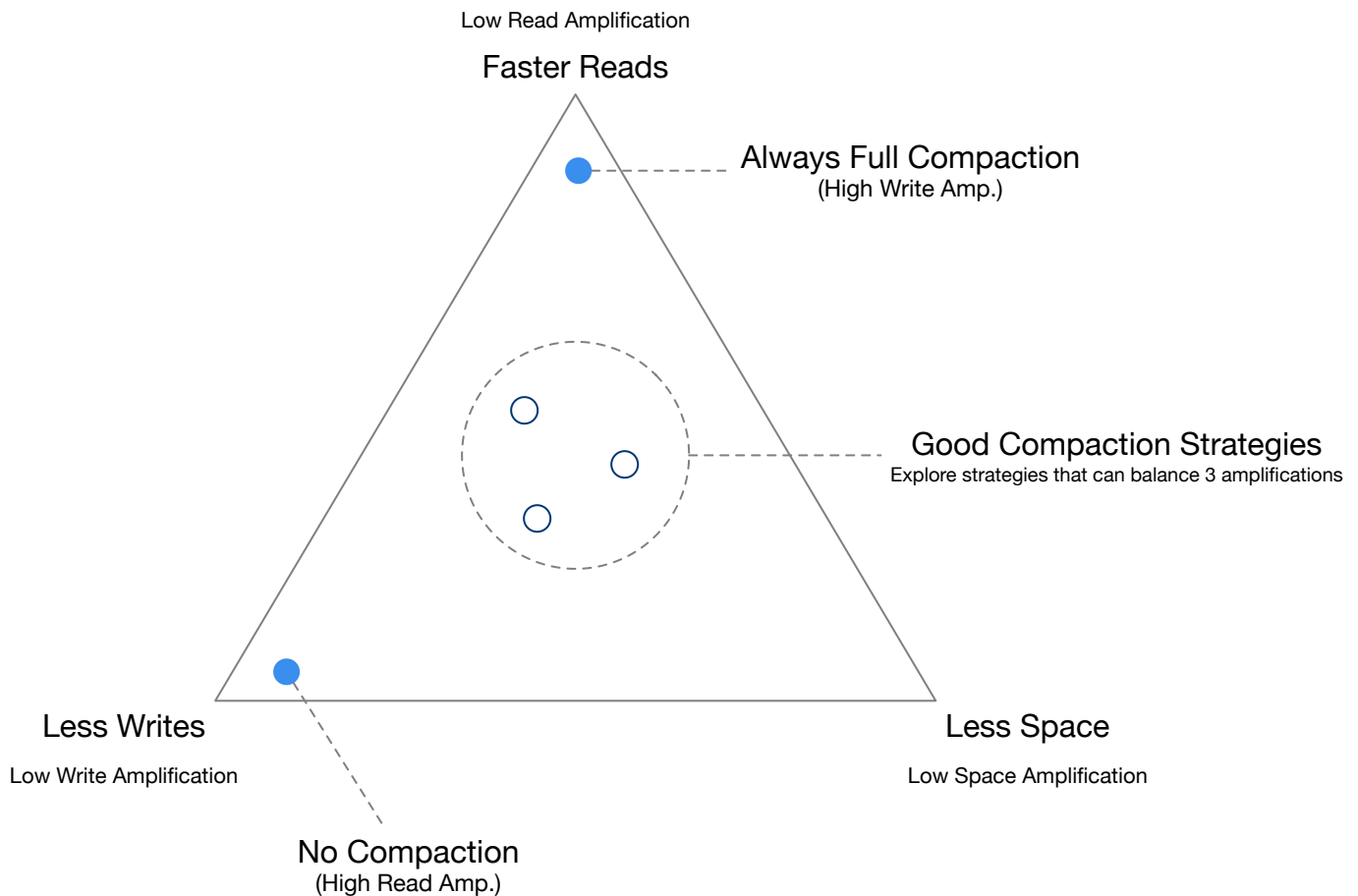
So from the above example, we have 2 naive ways of handling the LSM structure – not doing compactions at all, and always do full compaction when new SSTs are flushed.

Compaction is a time-consuming operation. It will need to read all data from some files, and write the same amount of files to the disk. This operation takes a lot of CPU resources and I/O resources. Not doing compactions at all leads to high read amplification, but it does not need to write new files. Always doing full compaction reduces the read amplification, but it will need to constantly rewrite the files on the disk.



The ratio of memtables flushed to the disk versus total data written to the disk is write amplification. That is to say, no compaction has a write amplification ratio of 1x, because once the SSTs are flushed to the disk, they will stay there. Always doing compaction has a very high write amplification. If we do a full compaction every time we get an SST, the data written to the disk will be quadratic to the number of SSTs flushed. For example, if we flushed 100 SSTs to the disk, we will do compactions of 2 files, 3 files, ..., 100 files, where the actual total amount of data we wrote to the disk is about 5000 SSTs. The write amplification after writing 100 SSTs in this cause would be 50x.

A good compaction strategy can balance read amplification, write amplification, and space amplification (we will talk about it soon). In a general-purpose LSM storage engine, it is generally impossible to find a strategy that can achieve the lowest amplification in all 3 of these factors, unless there are some specific data pattern that the engine could use. The good thing about LSM is that we can theoretically analyze the amplifications of a compaction strategy and all these things happen in the background. We can choose compaction strategies and dynamically change some parameters of them to adjust our storage engine to the optimal state. Compaction strategies are all about tradeoffs, and LSM-based storage engine enables us to select what to be traded at runtime.



One typical workload in the industry is like: the user first batch ingests data into the storage engine, usually gigabytes per second, when they start a product. Then, the system goes live and users start doing small transactions over the system. In the first phase, the engine should be able to quickly ingest data, and therefore we can use a compaction strategy that minimize write amplification to accelerate this process. Then, we adjust the parameters of the compaction algorithm to make it optimized for read amplification, and do a full compaction to reorder existing data, so that the system can run stably when it goes live.

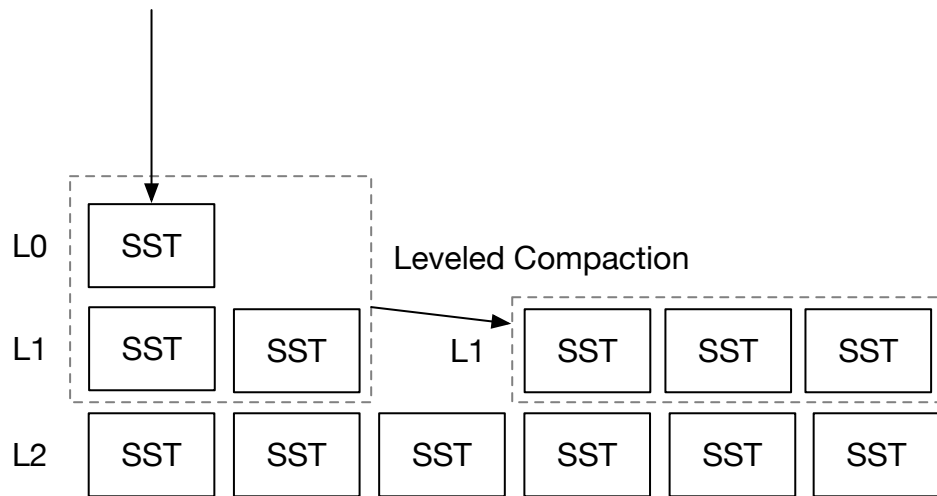
If the workload is like a time-series database, it is possible that the user always populate and truncate data by time. Therefore, even if there is no compaction, these append-only data can still have low amplification on the disk. Therefore, in real life, you should watch for patterns or specific requirements from the users, and use these information to optimize your system.

Compaction Strategies Overview

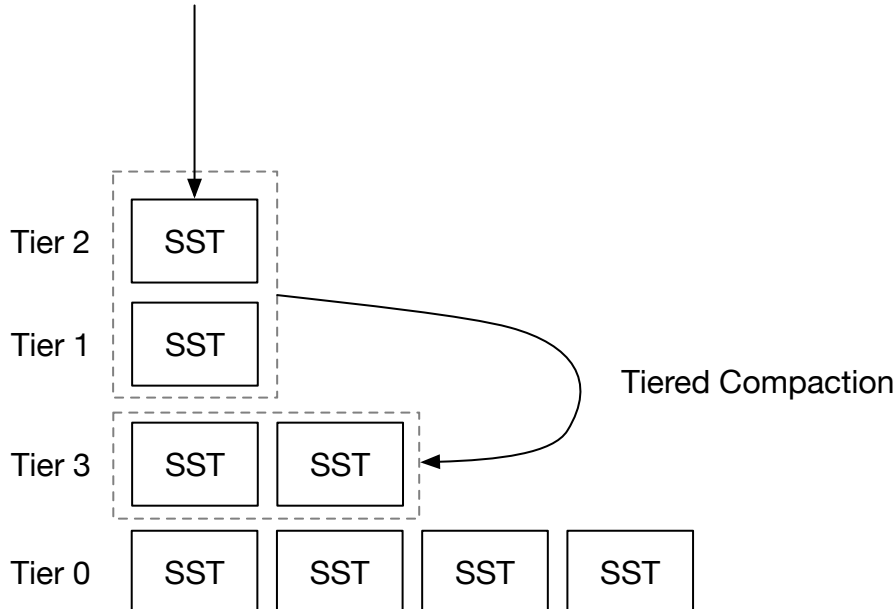
Compaction strategies usually aim to control the number of sorted runs, so as to keep read amplification in a reasonable amount of number. There are generally two categories of compaction strategies: leveled and tiered.

In leveled compaction, the user can specify a maximum number of levels, which is the number of sorted runs in the system (except L0). For example, RocksDB usually keeps 6 levels (sorted runs) in leveled compaction mode. During the compaction process, SSTs from

two adjacent levels will be merged and then the produced SSTs will be put to the lower level of the two levels. Therefore, you will usually see a small sorted run merged with a large sorted run in leveled compaction. The sorted runs (levels) grow exponentially in size – the lower level will be `<some number>` of the upper level in size.



In tiered compaction, the engine will dynamically adjust the number of sorted runs by merging them or letting new SSTs flushed as new sorted run (a tier) to minimize write amplification. In this strategy, you will usually see the engine merge two equally-sized sorted runs. The number of tiers can be high if the compaction strategy does not choose to merge tiers, therefore making read amplification high. In this course, we will implement RocksDB's universal compaction, which is a kind of tiered compaction strategy.



Space Amplification

The most intuitive way to compute space amplification is to divide the actual space used by the LSM engine by the user space usage (i.e., database size, number of rows in the database, etc.) . The engine will need to store delete tombstones, and sometimes multiple version of

the same key if compaction is not happening frequently enough, therefore causing space amplification.

On the engine side, it is usually hard to know the exact amount of data the user is storing, unless we scan the whole database and see how many dead versions are there in the engine. Therefore, one way of estimating the space amplification is to divide the full storage file size by the last level size. The assumption behind this estimation method is that the insertion and deletion rate of a workload should be the same after the user fills the initial data. We assume the user-side data size does not change, and therefore the last level contains the snapshot of the user data at some point, and the upper levels contain new changes. When compaction merges everything to the last level, we can get a space amplification factor of 1x using this estimation method.

Note that compaction also takes space – you cannot remove files being compacted before the compaction is complete. If you do a full compaction of the database, you will need free storage space as much as the current engine file size.

In this part, we will have a compaction simulator to help you visualize the compaction process and the decision of your compaction algorithm. We provide minimal test cases to check the properties of your compaction algorithm, and you should watch closely on the statistics and the output of the compaction simulator to know how well your compaction algorithm works.

Persistence

After implementing the compaction algorithms, we will implement two key components in the system: manifest, which is a file that stores the LSM state, and WAL, which persists memtable data to the disk before it is flushed as an SST. After finishing these two components, the storage engine will have full persistence support and can be used in your products.

If you do not want to dive too deep into compactions, you can also finish chapter 2.1 and 2.2 to implement a very simple leveled compaction algorithm, and directly go for the persistence part. Implementing full leveled compaction and universal compaction are not required to build a working storage engine in week 2.

Snack Time

After implementing compaction and persistence, we will have a short chapter on implementing the batch write interface and checksums.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.